

Azure application architecture fundamentals

A well-designed cloud application addresses reliability, security, cost, operations, and performance considerations. These considerations align with the business requirements, the specific characteristics of the cloud hosting platform, and the functionality that the platform provides.

You don't need to use a specific application style, such as microservices, to design an application for cloud workloads. However, cloud hosting makes many application design patterns more approachable than hosting solutions that don't natively provide a diverse selection of application and data platform options, scaling capabilities, security controls, and messaging options. Cloud workloads benefit from applications that are decomposed into smaller, decentralized services by design. These services communicate through APIs or by using asynchronous messaging or eventing. Applications scale horizontally by adding new instances when demand increases.

Applications that use the cloud's application hosting platforms, messaging capabilities, and decomposed services are subject to common concerns for distributed systems. In these systems, the application state is distributed, and operations are performed in parallel and asynchronously. Applications must be resilient when failures occur. Malicious actors continuously target applications. Deployments must be automated and predictable. Monitoring and telemetry are crucial for gaining insight into the system.

The following columns list some common characteristics of on-premises design and cloud design.

Typical on-premises design

- Monolithic and colocated functionality and data
- Designed for predictable scale or is overprovisioned
- Relational database
- Synchronized processing
- Designed to avoid failures and measures the mean time between failures (MTBF)
- Resources are provisioned through IT functions
- Snowflake servers and pet servers

Typical cloud design

- Decomposed and distributed functionality and data
- Designed for elastic scale
- Polyglot persistence by using a mix of storage technologies
- Asynchronous processing
- Designed to withstand malfunctions and measures MTBF
- Prepared for failure and measures the mean time to repair
- Resources are provisioned as needed through infrastructure as code
- Immutable and replaceable infrastructure

Design applications for Azure

Cloud architects who have expertise in cloud hosting and can make strategic tradeoff decisions should design cloud applications. Azure provides resources to help architects develop applications and guide development teams to implement them. To achieve good workload and application design, architects need to:

- [Align to organizational cloud adoption standards.](#)
- [Ensure that the design follows the Azure Well-Architected Framework.](#)
- Understand typical [architecture styles](#), [workloads](#), and [best practices](#).
- [Use design patterns to solve common problems and introduce strategic tradeoffs.](#)
- [Make informed technology choices.](#)
- [Evaluate reference architectures.](#)
- [Review service-specific guides.](#)

You can use Azure to host and rehost applications that aren't designed for the cloud. You can adjust workload applications to use cloud functionality, but rehosting an application that's designed for fixed resources and scale isn't considered a cloud-native deployment.

Align to organizational cloud adoption standards

Your application is part of a workload that likely needs to meet organizational standards and governance. Organizations of any size and cloud maturity should use the [Cloud Adoption Framework for Azure](#) to formalize their Azure-wide adoption strategy, readiness, innovation, management, governance, and security initiatives. Part of that approach is to standardize a

consistent approach across workloads, such as using [Azure landing zones](#). The Azure landing zone pattern provides organization-wide governance and gives workload teams and architects democratized access to resources to fulfill localized business objectives. As an architect who designs applications, it's crucial that you understand the macro environment and expectations for workload operations, such as application landing zones.

Your organization's Azure adoption strategy shouldn't affect the architectural style that you choose, but it might constrain technology choices or security boundaries.

Follow the Well-Architected Framework

You can evaluate any workload's design and implementation through various lenses. Use the Well-Architected Framework to evaluate and align your decisions with design principles across these five key architectural pillars:

- [Reliability](#)
- [Security](#)
- [Cost Optimization](#)
- [Operational Excellence](#)
- [Performance Efficiency](#)

By following these principles and evaluating the tradeoffs between these architectural pillars, you can produce a design that meets business requirements and is sufficiently durable, maintainable, secure, and cost optimized to run in Azure. These decisions should inform your architectural style choice and help narrow your technology choices or security boundaries as they relate to your specific workload's needs.

Your team or organization might have other design principles, such as [sustainability](#) and [responsible engineering](#), that you can use to evaluate your workload.

Understand typical architecture styles

After you understand the organizational environment that your application will exist in and the foundation of good architecture design based on the Well-Architected Framework, you need to decide what kind of architecture to build. It might be a microservices architecture, a more traditional N-tier application, or a big data solution. These architectural styles are distinct and

designed for different outcomes. When you evaluate architectural styles, you should also select data store models to address state management.

Evaluate the various [architecture styles](#) and [data store models](#) to understand the benefits and challenges that each option presents.

Workloads in the Well-Architected Framework

The [Well-Architected Framework workloads](#) provides perspectives of different workload classifications or types. For example, you can find articles about [mission-critical workloads](#), [AI and machine learning workloads](#), or [software-as-a-service workloads](#). These workload-specific articles apply the five core pillars of the Well-Architected Framework to the specific domain. If your application is part of a workload that aligns with one of these documented patterns, review the respective guidance to help you approach your design by following a set of workload-specific design principles and recommendations across common design areas like application platform, data platform, and networking. Some workload types might benefit from selecting a specific architectural style or data store model.

Best practices

For more information about various design considerations, including API design, autoscaling, data partitioning, and caching, see [Best practices in cloud applications](#). Review these considerations and apply the best practices that are appropriate for your application.

Use design patterns to solve common problems and introduce strategic tradeoffs

Your application has specific business requirements, goals, and measurements of success. You should decompose those functional and nonfunctional requirements into discrete activities that work together to achieve a solution that meets your and your customers' expectations. These activities typically follow patterns that the software industry has established. Software design patterns are named and repeatable approaches that you can apply to processing or data storage. These patterns are proven to solve specific problems with known tradeoffs.

The [catalog of cloud design patterns](#) in Azure addresses specific challenges in distributed systems. For AI workloads that include multiple autonomous agents, see [AI agent orchestration](#)

[patterns](#). It includes specialized coordination approaches that complement traditional design patterns by addressing the unique challenges of orchestrating intelligent, autonomous components.

Make informed technology choices

After you determine the type of architecture that you want to build and the design patterns that you expect to use, you can choose the main technology components for the architecture. The following technology choices are essential:

- *Compute* refers to the hosting model for the computing resources, or application platform, that your applications run on. For more information, see [Choose a compute service](#). See specialized guidance, like [Choose an Azure container service](#) and [Azure hybrid options](#), for specific application platforms.
- *Data stores* include databases and storage for files, caches, logs, and anything else that an application might persist to storage. For more information, see [Understand data store models](#) and [Review your storage options](#).
- *Messaging* technologies enable asynchronous messaging between components of the system. For more information, see [Asynchronous messaging options](#).
- *AI* technologies solve problems that are computationally complex to implement in traditional application code. For more information, see [Choose an AI services technology](#).

You'll probably make other technology choices along the way, but compute, data, messaging, and AI are central to most cloud applications and determine many aspects of your design.

Evaluate reference architectures

Azure Architecture Center is home to articles about solution ideas, example workloads, and reference architectures. These articles typically list the common components and considerations that align with the Well-Architected Framework. Some of these articles include a deployable solution that's hosted on GitHub. Although it's unlikely that any of these scenarios are exactly what you're building, they're a good starting point. You can adapt the guidance to your specific needs.

Browse the [catalog of architectures](#) in the Azure Architecture Center.

Review service-specific guides

After you select the core technology and consult the reference architectures, review documentation and guidance that's specific to the services in your architecture. Use the following resources for service-specific guidance:

- [Well-Architected Framework service guides](#): The Well-Architected Framework provides articles about many Azure services. The articles apply the five pillars of architecture to each service.
- [Azure reliability guides](#): The Azure reliability hub has in-depth articles that specifically address the reliability characteristics of many Azure services. These articles document some of the most critical reliability topics, such as availability zone support and expected behavior during different types of outages.

Coming from another cloud?

If you're familiar with how to design applications in another cloud provider, many of the same fundamentals apply. For example, architecture styles and cloud design patterns are conceptually cloud agnostic. For more information, see the following service mapping and architecture guide articles:

- [Azure for AWS professionals](#)
- [Azure for Google Cloud professionals](#)

For additional migration guidance, see the [Azure Migration Hub](#).

Next step

- [Architecture styles](#)

Last updated on 01/30/2026